

# THEKEDAR

ठेकेदार

## COMPLETE DEVELOPMENT ROADMAP

Phase 1 to 6 · Every File · Every Command · Every Route

|                |                   |                  |                      |                        |                 |
|----------------|-------------------|------------------|----------------------|------------------------|-----------------|
| Phase 1: Setup | Phase 2: Database | Phase 3: Backend | Phase 4: Customer UI | Phase 5: Contractor UI | Phase 6: Deploy |
|----------------|-------------------|------------------|----------------------|------------------------|-----------------|

### HOW TO USE THIS DOCUMENT

- Follow phases in order — each phase builds on the last. Do not skip.
- Complete every task top-to-bottom before moving to the next phase.
- Every code block is copy-paste ready. Use it exactly as shown.
- When stuck: paste the error + which Phase/Task to Claude and ask for help.

## OVERVIEW — 6 PHASES

| Ph | Name                  | What Gets Built                                                                                      | Time       | Goal                           |
|----|-----------------------|------------------------------------------------------------------------------------------------------|------------|--------------------------------|
| 1  | Foundation & Setup    | Git, Node.js, VS Code, Supabase account, folder structure, .env file, all software installed         | 3–5 days   | Codebase runs locally          |
| 2  | Database              | 10 PostgreSQL tables, Row Level Security, GPS indexes, DB connection from Node.js, seed data         | 4–6 days   | DB answers queries             |
| 3  | Backend API           | 35+ Express routes: auth/OTP/JWT, contractor CRUD, GPS search, reviews, file upload, admin, security | 12–18 days | Postman tests pass             |
| 4  | Frontend – Customer   | Connect homepage + search + profiles + reviews to live API. WhatsApp links, login modal, GPS search. | 8–12 days  | Customer flow works end-to-end |
| 5  | Frontend – Contractor | Register wizard, dashboard, availability toggle, portfolio upload — all connected to backend         | 6–10 days  | Contractor can self-register   |
| 6  | Deploy & Launch       | Vercel (frontend), Railway.app (backend), domain, SSL, admin panel, monitoring, analytics            | 4–7 days   | Live on the internet           |

## 1

## PHASE 1 — FOUNDATION & TOOLS

Install software · Create project · Git · Supabase account · Environment variables

GOAL: Your computer is set up, project folder exists, Git is tracking it, Supabase DB is created (empty).  
Duration: 3–5 days. No code yet — only setup.

### 1.1 Install These Tools (one time only)

| #   | Task                      | Details & Commands                                                                                  | Type    | Needs |
|-----|---------------------------|-----------------------------------------------------------------------------------------------------|---------|-------|
| 1.1 | Install Node.js           | nodejs.org → Download LTS (v20+) → Install<br>→ in Terminal run: node --version → should show v20.x | Must Do |       |
| 1.2 | Install VS Code           | code.visualstudio.com → Download → Install<br>→ Open                                                | Must Do |       |
| 1.3 | Install Git               | git-scm.com → Download → Install → in Terminal: git --version                                       | Must Do |       |
| 1.4 | Install Postman           | postman.com → Download Desktop App → Install (you will use this to test every API route)            | Must Do |       |
| 1.5 | VS Code Extensions        | Open VS Code → Extensions → Install: ESLint, Prettier, GitLens, REST Client                         | Config  | 1.2   |
| 1.6 | Create GitHub account     | github.com → Sign up → Verify email → this stores your code safely online                           | Must Do | 1.3   |
| 1.7 | Create Supabase account   | supabase.com → Sign up → New Project → Name: thekedaar → Region: Asia (Mumbai) → Save password!     | Must Do |       |
| 1.8 | Create Cloudinary account | cloudinary.com → Sign up free → copy Cloud Name, API Key, API Secret to notepad                     | Must Do |       |
| 1.9 | Create MSG91 account      | msg91.com → Sign up → complete KYC → get Auth Key → create OTP template                             | Must Do |       |

### 1.2 Create Project Folder Structure

Open Terminal (Mac: Cmd+Space → Terminal | Windows: Start → cmd) and run:

```
mkdir thekedaar && cd thekedaar
mkdir frontend backend
cd backend
npm init -y
npm install express cors helmet dotenv bcryptjs jsonwebtoken multer axios pg express-rate-limit sanitize-html
npm install --save-dev nodemon
cd ..
```

## 1.3 Folder Structure (will look like this when done)

```
thekedaar/
├── backend/
│   ├── src/
│   │   ├── routes/    ← API endpoints (auth.js, contractors.js, reviews.js, admin.js)
│   │   ├── middleware/ ← auth check, rate limiter, file validator
│   │   ├── controllers/ ← business logic for each route
│   │   ├── db/         ← pool.js (database connection)
│   │   └── utils/      ← otp.js, upload.js, validators.js
│   ├── .env            ← SECRET — never commit to Git!
│   ├── .env.example    ← commit this — shows what vars are needed
│   ├── .gitignore
│   ├── package.json
│   └── server.js       ← main entry point
├── frontend/
│   ├── index.html      ← homepage (already built)
│   ├── search.html
│   ├── profile.html
│   ├── login.html
│   ├── register.html
│   ├── contractor/
│   │   ├── register.html
│   │   └── dashboard.html
│   └── js/
│       ├── api.js      ← all fetch() calls go here
│       ├── auth.js
│       ├── search.js
│       └── profile.js
```

| #    | Task              | Details & Commands                                                                    | Type    | Needs |
|------|-------------------|---------------------------------------------------------------------------------------|---------|-------|
| 1.10 | Create .gitignore | In backend/ create file: .gitignore — add lines: node_modules/ .env *.log             | Must Do | 1.3   |
| 1.11 | Create .env file  | In backend/ create .env — content in Phase 2. NEVER commit this to GitHub!            | Config  | 1.7   |
| 1.12 | Initialize Git    | In thekedaar/ run: git init → git add . → git commit -m 'initial setup'               | Must Do | 1.6   |
| 1.13 | Connect to GitHub | On GitHub: New Repository → name: thekedaar → copy commands shown → paste in Terminal | Must Do | 1.12  |

## 2

## PHASE 2 — DATABASE LAYER

PostgreSQL schema · 10 tables · RLS · GPS indexes · DB connection

GOAL: All 10 database tables exist in Supabase, Row Level Security is ON, GPS indexes work, Node.js can query data. Duration: 4–6 days. Run SQL in Supabase → SQL Editor tab.

### 2.1 .env File — All Variables

Create backend/.env with these exact variable names:

```
# Server
PORT=5000
NODE_ENV=development

# Supabase (get from: Supabase → Settings → API)
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_ANON_KEY=your_anon_key_here
SUPABASE_SERVICE_KEY=your_service_role_key_here
DATABASE_URL=postgresql://postgres:[YOUR_PASSWORD]@db.xxxx.supabase.co:5432/postgres

# JWT (make up a random 64-character string)
JWT_SECRET=your_super_long_random_secret_minimum_64_characters_long
JWT_EXPIRES_IN=7d

# MSG91 (get from msg91.com dashboard)
MSG91_AUTH_KEY=your_msg91_auth_key
MSG91_TEMPLATE_ID=your_otp_template_id
MSG91_SENDER_ID=THKDR

# Cloudinary (get from cloudinary.com dashboard)
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# Frontend (for CORS — change to Vercel URL after deploy)
FRONTEND_URL=http://localhost:3000
```

### 2.2 Run These SQL Blocks in Supabase → SQL Editor

#### users table

```
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(200) NOT NULL,
  phone       VARCHAR(15) UNIQUE NOT NULL,
  email       VARCHAR(255) UNIQUE,
  password_hash VARCHAR(255),
  role        VARCHAR(20) DEFAULT 'customer' CHECK (role IN ('customer','contractor','admin')),
```

```
is_active    BOOLEAN DEFAULT true,  
created_at   TIMESTAMPTZ DEFAULT NOW(),  
updated_at   TIMESTAMPTZ DEFAULT NOW()  
);
```

### otps table

```
CREATE TABLE otps (  
  id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  phone     VARCHAR(15) NOT NULL,  
  code      VARCHAR(6) NOT NULL,  
  expires_at TIMESTAMPTZ NOT NULL,  
  used      BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX idx_otps_phone ON otps(phone);
```

### contractors table (main table — most important)

```
CREATE TABLE contractors (  
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id       UUID REFERENCES users(id) ON DELETE CASCADE,  
  business_name VARCHAR(300),  
  category      VARCHAR(100) NOT NULL,  
  description    TEXT,  
  daily_rate     INTEGER,  
  experience_years INTEGER DEFAULT 0,  
  team_size      INTEGER DEFAULT 1,  
  is_labour_group BOOLEAN DEFAULT false,  
  is_responsibility_model BOOLEAN DEFAULT false,  
  is_verified    BOOLEAN DEFAULT false,  
  is_featured    BOOLEAN DEFAULT false,  
  is_available   BOOLEAN DEFAULT true,  
  photo_url      TEXT,  
  portfolio_photos TEXT[] DEFAULT '{}',  
  services       TEXT[] DEFAULT '{}',  
  location_text  VARCHAR(500),  
  lat            DECIMAL(10,8),  
  lng            DECIMAL(11,8),  
  rating         DECIMAL(3,2) DEFAULT 0,  
  review_count   INTEGER DEFAULT 0,  
  views_count    INTEGER DEFAULT 0,  
  leads_count    INTEGER DEFAULT 0,  
  created_at     TIMESTAMPTZ DEFAULT NOW(),  
  updated_at     TIMESTAMPTZ DEFAULT NOW()  
);  
  
-- GPS index (needed for fast location search):  
CREATE INDEX idx_contractors_location ON contractors USING GIST (point(lng, lat));  
CREATE INDEX idx_contractors_category ON contractors(category);  
CREATE INDEX idx_contractors_rating ON contractors(rating DESC);
```

### remaining tables (reviews, sessions, reports, notifications)

```
CREATE TABLE reviews (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  contractor_id  UUID REFERENCES contractors(id) ON DELETE CASCADE,  
  reviewer_id   UUID REFERENCES users(id),  
  rating        INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),  
  comment       TEXT,  
  created_at    TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE TABLE sessions (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,  
  refresh_token TEXT UNIQUE NOT NULL,  
  expires_at  TIMESTAMPTZ NOT NULL  
);
```

```
CREATE TABLE reports (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  reporter_id  UUID REFERENCES users(id),  
  contractor_id UUID REFERENCES contractors(id),  
  reason       TEXT,  
  status       VARCHAR(30) DEFAULT 'pending',  
  created_at   TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE TABLE notifications (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,  
  message     TEXT NOT NULL,  
  type        VARCHAR(50),  
  is_read     BOOLEAN DEFAULT false,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

## 2.3 Row Level Security — Run This

```
-- Enable RLS (required for security):  
ALTER TABLE users      ENABLE ROW LEVEL SECURITY;  
ALTER TABLE contractors ENABLE ROW LEVEL SECURITY;  
ALTER TABLE reviews   ENABLE ROW LEVEL SECURITY;  
  
-- Anyone can read contractors (public search):  
CREATE POLICY contractors_public_read ON contractors FOR SELECT USING (true);  
  
-- Only owner can update their profile:  
CREATE POLICY contractors_owner_update ON contractors FOR UPDATE USING (auth.uid() = user_id);  
  
-- Anyone can read and write reviews:  
CREATE POLICY reviews_read ON reviews FOR SELECT USING (true);  
CREATE POLICY reviews_insert ON reviews FOR INSERT WITH CHECK (true);
```

## 2.4 Database Connection File

---

Create: backend/src/db/pool.js

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false }, // required for Supabase
  max: 10,
  idleTimeoutMillis: 30000,
});

module.exports = pool;

// To test: run in terminal from backend/ folder:
// node -e "const p=require('./src/db/pool'); p.query('SELECT NOW()').then(r=>console.log('DB OK:',r.rows[0])).catch(console.error)"
```

## 3

# PHASE 3 — BACKEND API (Node.js + Express)

35+ routes · Auth/OTP/JWT · GPS Search · File Upload · Security

GOAL: A fully working API that your frontend can call. All routes tested in Postman before touching the frontend. Duration: 12–18 days. Build one route group at a time.

TEST RULE: Every route group must pass Postman tests before moving on.

## 3.1 Main server.js File

Create: backend/server.js

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

const app = express();

// Security headers
app.use(helmet());

// CORS — only allow your frontend
app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));

// Rate limiting
const limiter = rateLimit({ windowMs: 15*60*1000, max: 100 });
const otpLimiter = rateLimit({ windowMs: 10*60*1000, max: 5 });
app.use('/api/', limiter);
app.use('/api/auth/send-otp', otpLimiter);

// Body parser (10kb max stops large payload attacks)
app.use(express.json({ limit: '10kb' }));

// Routes
app.use('/api/auth', require('./src/routes/auth'));
app.use('/api/contractors', require('./src/routes/contractors'));
app.use('/api/reviews', require('./src/routes/reviews'));
app.use('/api/admin', require('./src/routes/admin'));

// Health check
app.get('/health', (req, res) => res.json({ status: 'ok', time: new Date() }));

// 404 handler
```



```

app.use((req, res) => res.status(404).json({ error: 'Route not found' }));

// Global error handler
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal server error' });
});

app.listen(process.env.PORT || 5000, () => console.log('Server running'));

```

## 3.2 All Routes — Build In This Order

| #    | Task                             | Details & Commands                                                                                                | Type  | Needs |
|------|----------------------------------|-------------------------------------------------------------------------------------------------------------------|-------|-------|
| 3.1  | <b>POST /auth/send-otp</b>       | Validate phone → generate 6-digit OTP → save to otps table with 10-min expiry → call MSG91 API → return success   | Build | 2.4   |
| 3.2  | <b>POST /auth/verify-otp</b>     | Check phone+otp in DB → not expired → not used → mark used → find or create user → sign JWT → set httpOnly cookie | Build | 3.1   |
| 3.3  | <b>POST /auth/register</b>       | Hash password (bcrypt 12 rounds) → insert user → create empty contractor profile if role=contractor → return JWT  | Build | 3.1   |
| 3.4  | <b>POST /auth/login</b>          | Find user by email → bcrypt.compare password → sign JWT → return user (never return password_hash!)               | Build | 3.3   |
| 3.5  | <b>POST /auth/logout</b>         | Clear httpOnly cookie → delete session row from DB                                                                | Build | 3.2   |
| 3.6  | <b>GET /auth/me</b>              | Verify JWT from cookie/header → return user object (used on page load to restore login state)                     | Build | 3.3   |
| 3.7  | <b>Auth Middleware</b>           | Create src/middleware/auth.js → reads JWT from cookie or Authorization header → verifies → sets req.user          | Build | 3.3   |
| 3.8  | <b>Role Middleware</b>           | Create src/middleware/role.js → checkRole('admin') function → use on all /admin/* routes                          | Build | 3.7   |
| 3.9  | <b>GET /contractors/search</b>   | Accept: q, category, lat, lng, sort, verified, labour_group → SQL with Haversine GPS formula → paginated results  | Build | 2.4   |
| 3.10 | <b>GET /contractors/featured</b> | Return contractors where is_featured=true (active listing)                                                        | Build | 3.9   |
| 3.11 | <b>GET /contractors/:id</b>      | Return full profile → increment views_count                                                                       | Build | 3.9   |
| 3.12 | <b>GET /contractors/me</b>       | Auth required → return logged-in contractor's own profile                                                         | Build | 3.7   |
| 3.13 | <b>POST /contractors</b>         | Auth required (contractor role) → insert into contractors table → return created profile                          | Build | 3.7   |

| #    | Task                                       | Details & Commands                                                                                         | Type  | Needs |
|------|--------------------------------------------|------------------------------------------------------------------------------------------------------------|-------|-------|
| 3.14 | <b>PUT /contractors/me</b>                 | Auth required → update contractor's own DB row → sanitize all text with sanitize-html library              | Build | 3.12  |
| 3.15 | <b>PATCH /contractors/me/availability</b>  | Auth required → flip is_available boolean → return new value                                               | Build | 3.12  |
| 3.16 | <b>POST /contractors/photo</b>             | Auth + multer upload → validate MIME type (not just extension) → max 5MB → upload to Cloudinary → save URL | Build | 3.12  |
| 3.17 | <b>POST /contractors/portfolio</b>         | Auth + multer multi-file → max 5 files → Cloudinary upload all → save URLs array to DB                     | Build | 3.16  |
| 3.18 | <b>GET /reviews/contractor/:id</b>         | Return all reviews for contractor, newest first, with reviewer name                                        | Build | 2.4   |
| 3.19 | <b>POST /reviews/contractor/:id</b>        | Auth required → check not already reviewed → insert → recalculate avg rating → update contractor row       | Build | 3.11  |
| 3.20 | <b>GET /admin/contractors/pending</b>      | Admin auth → return unverified contractors sorted by registration date                                     | Build | 3.8   |
| 3.21 | <b>PATCH /admin/contractors/:id/verify</b> | Admin auth → set is_verified=true → create notification for contractor                                     | Build | 3.20  |
| 3.22 | <b>GET /admin/stats</b>                    | Admin auth → return: total users, contractors, verified count, reviews today, active featured              | Build | 3.20  |
| 3.23 | <b>GET /admin/reports</b>                  | Admin auth → return all pending reported profiles                                                          | Build | 3.20  |

### 3.3 GPS Search — Haversine Formula (copy exactly)

Add this SQL to your GET /contractors/search route in Node.js:

```
const { q, category, lat, lng, sort } = req.query;

const sql = `
  SELECT *,
    ( 6371 * acos(
      cos(radians($1)) * cos(radians(lat))
      * cos(radians(lng) - radians($2))
      + sin(radians($1)) * sin(radians(lat))
    )) AS distance_km
  FROM contractors
  WHERE is_verified = true
  ${category ? 'AND category = $3' : ''}
  ORDER BY ${sort === 'rating' ? 'rating DESC' : 'distance_km ASC'}
  LIMIT 20 OFFSET ${category ? 4 : 3}
`;
```

```
const values = lat && lng ? [lat, lng, ...(category?[category]:[]), offset] : [0, 0, ...(category?[category]:[]), offset];
const result = await pool.query(sql, values);
```

### 3.4 JWT Auth Middleware (copy exactly)

Create: backend/src/middleware/auth.js

```
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  // Try cookie first, then Authorization header
  const token = req.cookies?.token || req.headers.authorization?.replace('Bearer ', '');

  if (!token) {
    return res.status(401).json({ error: 'Unauthorized — please log in' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { id, role, phone }
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
};
```

### 3.5 Postman Testing Checklist

| Test in Postman                             | Expected                                    | Pass / Fail                                                 |
|---------------------------------------------|---------------------------------------------|-------------------------------------------------------------|
| GET /health                                 | { status: 'ok' }                            | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| POST /auth/register (new user)              | { user: {...}, token: '...' } 201           | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| GET /auth/me with token in header           | { id, name, role }                          | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| POST /auth/send-otp (valid phone)           | { success: true } 200                       | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| POST /auth/verify-otp (wrong code)          | { error: 'Invalid OTP' } 400                | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| GET /contractors/search?category=electrical | Array of contractors with distance_km field | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| GET /contractors/:id (real UUID)            | Full contractor profile object              | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| POST /contractors (no auth)                 | { error: 'Unauthorized' } 401               | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| POST /reviews/:id (duplicate)               | { error: 'Already reviewed' } 409           | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |
| GET /admin/stats (no admin token)           | { error: 'Forbidden' } 403                  | <input type="checkbox"/> Pass <input type="checkbox"/> Fail |

GOAL: The frontend HTML you already have gets connected to your live backend. Duration: 8–12 days. Change one feature at a time. Test it. Then move on.

## 4.1 Create api.js — All API Calls Go Here

Create: frontend/js/api.js

```
const API_BASE = 'http://localhost:5000/api'; // change to Railway URL after deploy
```

```
async function apiCall(endpoint, options = {}) {
  try {
    const res = await fetch(API_BASE + endpoint, {
      credentials: 'include', // sends auth cookies
      headers: { 'Content-Type': 'application/json', ...options.headers },
      ...options,
    });
    if (!res.ok) {
      const err = await res.json();
      throw new Error(err.message || `Request failed ${res.status}`);
    }
    return await res.json();
  } catch (err) {
    console.error('API Error:', err.message);
    throw err;
  }
}

const Auth = {
  sendOTP: (phone) => apiCall('/auth/send-otp', { method: 'POST', body: JSON.stringify({phone}) }),
  verifyOTP: (phone,otp) => apiCall('/auth/verify-otp', { method: 'POST', body: JSON.stringify({phone,otp}) }),
  login: (data) => apiCall('/auth/login', { method: 'POST', body: JSON.stringify(data) }),
  logout: () => apiCall('/auth/logout', { method: 'POST' }),
  me: () => apiCall('/auth/me'),
  register: (data) => apiCall('/auth/register', { method: 'POST', body: JSON.stringify(data) }),
};

const Contractors = {
  search: (params) => apiCall('/contractors/search?' + new URLSearchParams(params)),
  featured: () => apiCall('/contractors/featured'),
  getById: (id) => apiCall('/contractors/' + id),
  getMyProfile: () => apiCall('/contractors/me'),
  create: (data) => apiCall('/contractors', { method: 'POST', body: JSON.stringify(data) }),
  update: (data) => apiCall('/contractors/me', { method: 'PUT', body: JSON.stringify(data) }),
  setAvailability: (v) => apiCall('/contractors/me/availability', { method: 'PATCH', body:
    JSON.stringify({available:v}) }),
};
```

```
const Reviews = {
  get: (id) => apiCall('/reviews/contractor/' + id),
  submit: (id, data) => apiCall('/reviews/contractor/' + id, { method: 'POST', body: JSON.stringify(data) }),
};
```

| #    | Task                                       | Details & Commands                                                                                      | Type   | Needs |
|------|--------------------------------------------|---------------------------------------------------------------------------------------------------------|--------|-------|
| 4.1  | <b>Homepage: real featured contractors</b> | Replace mock CONTRACTORS array → call Contractors.featured() → render same cards with live data         | Build  | 3.10  |
| 4.2  | <b>Search: real results</b>                | On form submit → call Contractors.search({q, category}) → replace renderContractors() output            | Build  | 4.1   |
| 4.3  | <b>GPS search</b>                          | reqLoc() gets coordinates → pass lat+lng to Contractors.search() → sort by distance                     | Build  | 4.2   |
| 4.4  | <b>Category clicks filter search</b>       | filterCat() → Contractors.search({category}) → update displayed cards                                   | Build  | 4.2   |
| 4.5  | <b>Create search.html page</b>             | Full search results page: filter panel, sort dropdown, contractor grid, pagination, empty state         | Build  | 4.3   |
| 4.6  | <b>Create profile.html page</b>            | Fetch by ID from URL param → render: photo, name, badges, rating, tabs (About/Portfolio/Reviews)        | Build  | 4.4   |
| 4.7  | <b>Load real reviews on profile</b>        | Reviews.get(id) → render star ratings + comments. Show 'No reviews yet' if empty.                       | Build  | 4.6   |
| 4.8  | <b>Submit review</b>                       | Form → check logged in → Reviews.submit(id, data) → toast success → reload reviews                      | Build  | 4.7   |
| 4.9  | <b>Login modal: real OTP</b>               | sendOTP() → Auth.sendOTP() → verifyOTP() → Auth.verifyOTP() → save user to localStorage → update navbar | Build  | 3.2   |
| 4.10 | <b>WhatsApp button: real phone</b>         | Replace hardcoded number with contractor.phone from API response                                        | Build  | 4.1   |
| 4.11 | <b>Loading states</b>                      | Show spinner while fetching → skeleton cards → empty state if 0 results                                 | Design | 4.2   |
| 4.12 | <b>Error states</b>                        | If API fails: show retry button → never show raw JSON errors to user                                    | Design | 4.11  |

## 5

## PHASE 5 — FRONTEND (CONTRACTOR SIDE)

Register wizard · Dashboard · Availability · Portfolio upload · Profile edit

GOAL: A contractor can register, manage their profile, toggle availability, and upload photos. Duration: 6–10 days.

| #    | Task                                     | Details & Commands                                                                                | Type   | Needs |
|------|------------------------------------------|---------------------------------------------------------------------------------------------------|--------|-------|
| 5.1  | <b>register-contractor.html (Step 1)</b> | Fields: name, phone, password → Auth.register({...role:'contractor'}) → save token → go to step 2 | Build  | 3.3   |
| 5.2  | <b>Register Step 2: Profile</b>          | Fields: category, description, daily_rate, experience_years → Contractors.create(data)            | Build  | 5.1   |
| 5.3  | <b>Register Step 3: Profile photo</b>    | File input → preview → FormData upload → POST /contractors/photo → show progress                  | Build  | 5.2   |
| 5.4  | <b>Register Step 4: ID proof</b>         | File upload → show 'Pending Verification - 24hrs' message → explain the review process            | Build  | 5.3   |
| 5.5  | <b>dashboard.html (protected)</b>        | Check localStorage for token → if none: redirect to login → show contractor profile + stats       | Build  | 5.4   |
| 5.6  | <b>Dashboard: live stats</b>             | Contractors.getMyProfile() → display: rating, review_count, views_count, is_verified badge        | Build  | 5.5   |
| 5.7  | <b>Availability toggle</b>               | Toggle button → Contractors.setAvailability(true/false) → update UI instantly                     | Build  | 5.6   |
| 5.8  | <b>Portfolio upload</b>                  | Multi-file input → max 5 JPG/PNG → max 5MB each → POST /contractors/portfolio                     | Build  | 5.7   |
| 5.9  | <b>Edit profile page</b>                 | Prefill current data → PUT /contractors/me on save → toast success                                | Build  | 5.6   |
| 5.10 | <b>Verification banner</b>               | If is_verified=false: show orange 'Pending Verification' banner with explanation                  | Design | 5.6   |

## 6

## PHASE 6 — DEPLOY &amp; LAUNCH

Railway.app · Vercel · Domain · SSL · Admin Panel · Monitoring

GOAL: Thekedaar is live on the internet. Anyone opens thekedaar.com on their phone and uses it.  
Duration: 4–7 days.

| #    | Task                                | Details & Commands                                                                                          | Type    | Needs |
|------|-------------------------------------|-------------------------------------------------------------------------------------------------------------|---------|-------|
| 6.1  | <b>Railway.app backend deploy</b>   | railway.app → New Project → Deploy from GitHub → select backend/ folder → set start command: node server.js | Deploy  |       |
| 6.2  | <b>Add env variables to Railway</b> | Railway → Variables tab → add ALL variables from your .env file one by one                                  | Config  | 6.1   |
| 6.3  | <b>Get Railway URL</b>              | Copy URL like:<br>https://thekedaar.up.railway.app — test: open /health in browser                          | Test    | 6.2   |
| 6.4  | <b>Update API_BASE in api.js</b>    | Change from http://localhost:5000 to your Railway URL before Vercel deploy                                  | Must Do | 6.3   |
| 6.5  | <b>Vercel frontend deploy</b>       | vercel.com → New Project → Import GitHub → Root: frontend/ → Deploy                                         | Deploy  | 6.4   |
| 6.6  | <b>Test full flow live</b>          | Open Vercel URL → search contractors → view profile → click WhatsApp → test login OTP                       | Test    | 6.5   |
| 6.7  | <b>Buy domain (optional)</b>        | namecheap.com → thekedaar.in ~₹700/yr → add to Vercel → update FRONTEND_URL in Railway env                  | Deploy  | 6.5   |
| 6.8  | <b>Build admin panel</b>            | admin.html: pending contractors list, verify button, reports queue, platform stats                          | Build   | 3.22  |
| 6.9  | <b>Set up UptimeRobot</b>           | uptimerobot.com (free) → monitor Railway URL → get WhatsApp alert if site goes down                         | Config  | 6.3   |
| 6.10 | <b>Add Google Analytics</b>         | GA4 → create property → add script tag to all HTML pages → track: searches, profile views, WA taps          | Config  | 6.5   |
| 6.11 | <b>Daily DB backup</b>              | Supabase → Settings → Database → turn on backups OR run manual pg_dump weekly                               | Config  | 6.2   |

# SECURITY CHECKLIST — Do Before Launch

Every item below is a real attack vector. If you skip any, your users' data is at risk.

|                          | Threat                    | What You Must Do                                                                                                                                                        | Where   |
|--------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <input type="checkbox"/> | <b>SQL Injection</b>      | NEVER build SQL strings with user input. Always use parameterised queries: <code>pool.query('SELECT * WHERE id=\$1', [id])</code>                                       | Backend |
| <input type="checkbox"/> | <b>XSS Attacks</b>        | Sanitize all user text with <code>sanitize-html</code> before saving. On frontend use <code>textContent</code> not <code>innerHTML</code> for user data.                | Both    |
| <input type="checkbox"/> | <b>Brute Force</b>        | Rate limit: 5 OTP requests per 10 min per phone. 10 login attempts per 15 min per IP.                                                                                   | Backend |
| <input type="checkbox"/> | <b>File Upload Attack</b> | Check MIME type with <code>file-type</code> library. Reject non-image. Max 5MB. Check filename for <code>../</code> path traversal.                                     | Backend |
| <input type="checkbox"/> | <b>JWT Theft</b>          | Store JWT in <code>httpOnly</code> cookie (NOT <code>localStorage</code> ). Set: <code>Secure=true</code> , <code>SameSite=Strict</code> , <code>HttpOnly=true</code> . | Backend |
| <input type="checkbox"/> | <b>Data Exposure</b>      | NEVER return <code>password_hash</code> in API responses. NEVER log OTP codes. NEVER commit <code>.env</code> to Git.                                                   | Backend |
| <input type="checkbox"/> | <b>CORS Attack</b>        | Only allow your Vercel URL in CORS. NEVER use <code>origin: '*'</code> in production.                                                                                   | Backend |
| <input type="checkbox"/> | <b>ID Proof Exposure</b>  | Store ID photos in Cloudinary private folder. Serve only via signed URLs. Never public access.                                                                          | Backend |



## TROUBLESHOOTING — Common Errors & Exact Fixes

| Error / Problem                | Exact Fix                                                                                                          |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Cannot find module 'express'   | cd backend → run: npm install — you are missing node_modules folder                                                |
| EADDRINUSE: port 5000 in use   | Run: npx kill-port 5000 OR change PORT=5001 in .env                                                                |
| Error connecting to database   | DATABASE_URL in .env is wrong. Go to Supabase → Settings → Database → copy string again                            |
| JWT malformed / invalid        | Token empty or corrupted. Open DevTools → Application → Cookies → check token exists. Check JWT_SECRET is correct. |
| CORS error in browser console  | FRONTEND_URL in Railway env must exactly match your Vercel URL including https:// and no trailing /                |
| OTP SMS not received           | Test MSG91 from their dashboard first. Check AUTH_KEY is correct. Check DLT registration done.                     |
| 413 Payload Too Large          | In server.js: app.use(express.json({ limit: '10mb' }))) and set multer limits to match                             |
| Supabase: violates RLS policy  | You are using anon key but RLS blocks it. Either use service_role key in backend OR adjust RLS policy              |
| GPS search returns empty       | Check lat/lng columns have decimal data. Run raw SQL in Supabase editor to test formula.                           |
| fetch failed (network error)   | Backend is not running. In backend/ folder run: nodemon server.js                                                  |
| Vercel deploy fails            | Check vercel.json in frontend/ folder. Plain HTML/JS needs no build step — set output dir to ./                    |
| Image not showing after upload | Cloudinary URL correct but image private. Set upload_preset to unsigned or use signed URL in response              |

## ACCOUNTS & SERVICES REQUIRED

| Service          | Cost              | What It Does                 | Sign Up URL                                                     |
|------------------|-------------------|------------------------------|-----------------------------------------------------------------|
| Supabase         | FREE (500MB)      | PostgreSQL database in cloud | <a href="https://supabase.com">supabase.com</a>                 |
| Railway.app      | FREE (\$5 credit) | Hosts Node.js backend        | <a href="https://railway.app">railway.app</a>                   |
| Vercel           | FREE forever      | Hosts HTML/CSS/JS frontend   | <a href="https://vercel.com">vercel.com</a>                     |
| MSG91            | ~₹0.15/OTP        | Sends OTP SMS to users       | <a href="https://msg91.com">msg91.com</a> (KYC required)        |
| Cloudinary       | FREE (25GB)       | Stores contractor photos     | <a href="https://cloudinary.com">cloudinary.com</a>             |
| GitHub           | FREE              | Stores code, enables deploys | <a href="https://github.com">github.com</a>                     |
| UptimeRobot      | FREE              | Alerts if site goes down     | <a href="https://uptimerobot.com">uptimerobot.com</a>           |
| Google Analytics | FREE              | Tracks user behaviour        | <a href="https://analytics.google.com">analytics.google.com</a> |

## HOW TO GET HELP — Sharing Code With Claude

When you hit an error, share it with Claude using one of these methods:

METHOD 1 — Single file (most common):

Open file in VS Code → Ctrl+A → Ctrl+C → paste in Claude chat

Say: 'Phase 3 Task 3.9 — getting this error: [error message]'

METHOD 2 — Multiple files at once (Windows):

Open VS Code Terminal (Ctrl+`) → type `src\routes\*.js` → copy output → paste

METHOD 3 — Show error log:

Copy the FULL error message from VS Code terminal → paste to Claude

ALWAYS INCLUDE: Phase number + Task number + exact error message

Example: 'Phase 3, Task 3.19, POST /reviews returns 500, here is the code and error'

### THEKEDAAR — ठेकेदार

Har Kaam Ka Ek Thekedaar

Follow the phases. Build one thing at a time. Every error has a fix.

Alok Kumar · MANIT Bhopal · 2025